SPARE: Securing Progressive Web Applications Against Unauthorized Replications

a,*, b, c

[a] *George Washington University, USA*
[b]
[c]

Abstract

WebView applications are widely used in mobile applications to display web content directly within the app, enhancing user engagement by eliminating the need to open an external browser and providing a seamless experience. Progressive Web Applications (PWAs) further improve usability by combining the accessibility of web apps with the speed, offline capabilities, and responsiveness of native applications. However, malicious developers can exploit this technology by duplicating PWA web links to create counterfeit native apps, thereby monetizing through user diversion. This unethical practice poses significant risks to both users and the original application developers, underscoring the need for robust security measures to prevent unauthorized replication. To the best of our knowledge, we are the first to address this issue and propose a practical solution to defend against or mitigate such attacks. We analyze the vulnerabilities of our proposed security solution to assess its effectiveness and introduce advanced measures to address any identified weaknesses, presenting a comprehensive defense framework. As part of our work, we developed a prototype web application that secures PWAs from replication by embedding a combination of Unix timestamps and device identifiers into the query parameters. We evaluate the effectiveness of this defense strategy by simulating an advanced attack scenario. Additionally, we created a realistic dataset reflecting mobile app user behavior, modeled using a Zipfian distribution, to validate our framework.

Keywords: Cyber-physical systems, internet of things, machine learning, cyberattacks, threat analysis, formal model.

1. Introduction

A Progressive Web Application (PWA) leverages web technologies to provide a user experience similar to a native application, making them particularly suitable for mobile devices due to their responsiveness and adaptability to various screen sizes. As the digital landscape continues to evolve, PWAs have emerged as a popular solution, combining the accessibility of web applications with the functionalities of native apps. By 2027, the PWA market is set to reach $10.44 billion with a growth rate of about 32% [1]. However, this popularity also brings increased scrutiny and potential security threats, particularly regarding the confidentiality of user information. A malicious developer can execute a replay attack by creating and distributing a WebView-based application that embeds the URL of a target PWA, deceiving users by displaying the actual web app within the WebView. Adversaries may have several motivations for creating such applications. Firstly, the malicious app can inject its own advertisements into the WebView, earning direct monetary benefits from victims while harming the reputation of the legitimate organization and causing financial losses. Secondly, the malicious app can intercept user information and exploit it for fraudulent activities. Third, it can deploy hidden malicious payloads to victims' devices, using WebView as a facade for broader attacks, such as distributing ransomware or spyware.

*Corresponding author

Although replay attacks using malicious WebView applications pose significant threats, we are not aware of any research efforts aimed at countering them. We propose a novel approach to secure PWAs by embedding dynamic query parameters into their URLs. By encrypting two key query parameters—Unix Time Frame and Device ID—on the Android side using symmetric encryption, we can significantly enhance the security posture of our application. The Unix Time Frame serves multiple purposes; it provides a constantly changing value that limits the validity of any copied URL, thereby rendering it ineffective shortly after its creation. This time-sensitive mechanism ensures that even if an adversary gains access to a URL, it will become invalid after a predetermined period. The Device ID, however, acts as an additional layer of security. In scenarios where adversaries attempt to create dynamic review applications by leveraging the original app, Device ID allows our backend to recognize and limit access based on specific devices. This strategy mitigates the risk of unauthorized access, as each Device ID is unique and tied to a particular user device. By employing these two parameters, our approach not only fortifies the security of the PWA but also ensures that sensitive information remains protected from standard exploitation methods. This paper will delve deeper into implementing this security mechanism, discuss its effectiveness against various attack vectors, and outline the potential implications for developers and users in the PWA ecosystem. Through this exploration, we aim to contribute to the ongoing discourse on enhancing security in web applications, ultimately fostering a safer digital environment for all users.

We devise a framework for protecting PWAs named **S**ecurity-Driven **P**rotection **A**gainst **R**eplication **E**xploits (SPARE).

The principal contribution of our work is threefold.

- We developed a PWA with added security features to protect against user diversion caused by malicious applications.
- We considered a realistic attack model and demonstrated a critical attack capable of evading the implemented security measures.
- We further strengthened the application by enforcing a robust security policy, evaluated the vulnerability of the enhanced application, and thus proposed a defense guide.

All implementation and evaluation results are reproducible with the source code on GitHub. The rest of the paper is organized as follows: we provide an overview of the PWA, webapps, and other background information in Section 2. We formally describe the problem domain and the attack model in Section 3. In the following section, we present the technical details of the proposed SPARE framework (4). We provide case studies to give insights about our proposed framework's working principle and capabilities in Section 5. We evaluate SPARE using our synthesized realistic datasets based on Zipfian distribution in Section 6. We conclude the paper in Section 9.

2. Background

A Progressive Web App (PWA) is a web-based application that leverages standard web technologies to deliver an experience comparable to a native app designed for a specific platform. A Progressive Web App (PWA) works across various devices and platforms using a single codebase, much like a traditional website. However, it also offers features typical of native apps, such as being installable, functioning offline or in the background, and integrating with the device and other apps. PWAs bring together the best aspects of websites and native applications.

Vulnerabilities: symmetric encryption AES

The PWA app has a standalone feature. It does not require any enforcement of any webview.

2.1. Background Study

Progressive Web Applications (PWAs) represent a revolutionary approach to web development, blending the universal accessibility of websites with the rich functionality traditionally associated with native mobile applications. By harnessing modern web technologies like service workers, manifest files, and HTTPS

protocols, PWAs deliver features that users typically expect only from installed applications—offline access, push notifications, home screen installation, and background data synchronization (Singh Singh, 2020). This versatility has made PWAs increasingly attractive to developers seeking to create cross-platform experiences without navigating the complexities of app store deployment processes. Yet, this very openness that makes PWAs so accessible across platforms also creates significant security vulnerabilities. Among the most concerning yet understudied threats is the unauthorized replication of PWAs through WebView-based applications. In these scenarios, malicious actors can develop mobile applications that simply embed a WebView component displaying a legitimate PWA's URL. This effectively clones the original application's functionality and interface without requiring access to its source code. These unauthorized clones are then distributed as standalone mobile applications, often injected with advertisements, tracking code, or phishing mechanisms—directly undermining the original developer's intellectual property, revenue streams, and user trust.

Progressive Web Applications (PWAs) have demonstrated increasing prevalence across digital platforms. Prominent social media entities, including Facebook, Instagram, X (formerly Twitter), and YouTube, have implemented PWA functionality. The integration of features such as application-like interfaces and cross-platform compatibility enables developers to engineer Android applications through the Trusted Web Activity (TWA) methodology, subsequently facilitating distribution via the Play Store ecosystem. PWAs operating within TWA architecture necessarily require browser infrastructure support, predominantly through Chrome Engine implementation. Although browsing histories are recorded within browser repositories, this presents a potential vulnerability wherein malicious actors may extract URLs from historical records. This vulnerability enables the unauthorized creation of webview applications.

While webview applications demonstrate limitations regarding offline functionality and native interface performance, primarily functioning as website containers within application frameworks, they can establish communication with frontend systems through JavaScript Bridge integration. Despite this capability representing a functional compromise, the extraction and replication of URLs remain viable for webview application development. The protection of PWA-based Android Applications (TWAs) from unauthorized duplication presents significant challenges. Currently, There is a bidirectional communication protocol using PostMessage which requires Chrome version 115.0.5790.13 or higher. Which means it requires at least Android Version 8. This communication is session-based. It means that the ability to send and receive messages between your TWA (Android) and PWA (web) exists only during the active session of the TWA. A CustomTabsSession binds the Android app and the browser. If the Chrome process is killed, or the TWA is backgrounded for too long, the session can be lost.

There is another communication method that operates unidirectionally (Android $\rightarrow$ Frontend), utilizing query parameters. While API implementation could potentially address this security concern, such solutions introduce server dependencies and potential performance degradation during PWA loading procedures, thereby compromising user experience and architectural simplicity. The fundamental objective of PWA development—simplified implementation methodology—would be compromised by such complexity.

Query parameter implementation represents an established unidirectional communication methodology. Our research introduces an approach to differentiate between legitimate application requests and potentially malicious access attempts through encrypted query parameter implementation.

2.2. Motivation

The recent surge in low-code and no-code development platforms, combined with inconsistent app store enforcement policies, has dramatically lowered the technical barriers for creating WebView wrappers around existing PWAs. This trend raises several serious concerns:

- **Brand Impersonation:** Bad actors can easily present themselves as official app providers, misleading users and potentially damaging the reputation of legitimate brands.
- **Monetization Theft:** Unauthorized wrapper applications frequently incorporate advertisements or tracking mechanisms that divert potential revenue away from the rightful developers.

- **User Security Compromises:** These wrapped PWAs may contain malicious code injections, phishing overlays, or invasive tracking tools that compromise end-user security and privacy.
- **Infrastructure Exploitation:** Replicated applications continue to rely on the legitimate application's backend systems, increasing server loads and operational costs without contributing to intended business metrics.

Despite these significant threats, current research literature and developer resources offer surprisingly limited guidance on detecting, preventing, or mitigating such attacks specifically targeting PWAs. The absence of standardized defense mechanisms leaves PWA developers vulnerable and forces them into reactive rather than proactive security postures.

This paper aims to address this critical gap by proposing a comprehensive, multi-layered defense strategy to shield PWAs from unauthorized WebView replication.

2.3. Literature Review

Progressive Web Apps (PWAs) are web applications that behave like native apps, offering offline support, push notifications, and installation to the home screen. Trusted Web Activities (TWAs) extend this by allowing PWAs to run in full-screen mode within Android apps, using Chrome Custom Tabs under the hood. Although these technologies improve cross-platform development and user experience, they expose the application to risks not typically associated with native apps, primarily due to their reliance on web technologies and browser containers.

The one-way communication using query parameters enables Trusted Web Activity(TWA) to communicate with the PWA. Due to browser history access in PWA apps, users can see the URL, making it easy to create webview apps by simply changing the URL. Attackers can display inappropriate ads on top of the webview to any age group, as their primary goal is to copy the main app to generate revenue. For financial transaction applications like ticket-buying websites, attackers can copy the PWA app and redirect users to malicious transaction sites from the top layer.

Digital Asset Links (DAL) add a layer of protection against attackers creating TWA apps. Without valid asset links, the address bar will remain visible at the top of attacker-made TWA apps. When an attacker's TWA doesn't match the fingerprint (SHA-256), the app launches with the address bar displayed. The developers of the Trusted Web Activity recommended using DAL to verify domain ownership in TWA. For two-way communication, JavaScript User Interface (JSUI) can work, but it presents security issues and requires manual caching of HTML/JS for offline support. Performance-wise, PWA is significantly better than WebView. Communication from a Progressive Web App frontend to its Android wrapper via Trusted Web Activity presents several challenges. While WebView supports bidirectional communication between JavaScript and native code, TWA lacks this capability. Despite TWA making it easy to bring PWAs to the Play Store, communication between the PWA and Android native shell remains restricted.

2.3.1. WebView Wrapping and Security Risks

The security implications of WebView components have been extensively investigated within the Android and iOS application ecosystems. Groundbreaking research by Enck et al. (2009) identified how WebView components can be exploited to inject malicious scripts or impersonate legitimate applications. Later studies by Kotzias et al. (2017) revealed that numerous applications published on official app stores are actually malicious wrappers of legitimate web content, created primarily to deliver unsolicited advertisements or harvest user data.

For PWAs specifically, this vulnerability is amplified due to their publicly accessible URL architecture. Sivaprasad et al. (2021) highlighted that while PWAs are typically designed with a domain-centric approach, they fundamentally lack binding mechanisms that would restrict their execution to trusted environments or containers. This architectural limitation makes them inherently susceptible to unauthorized embedding or misuse through WebView wrappers.

2.3.2. Existing Client-Side and Server-Side Defenses

Common client-side defenses such as Content Security Policy (CSP) and `X-Frame-Options` headers are frequently recommended to prevent framing attacks and unauthorized embedding. However, these protective measures were primarily designed to prevent iframe inclusion on websites and offer limited protection against mobile WebView-based embedding, particularly when WebView clients deliberately override or ignore these security policies [**?**].

Server-side protection techniques have also been explored, including User-Agent filtering, Origin and Referer header validation, and token-based access control systems. While these approaches can help identify unauthorized access patterns, sophisticated attackers can easily circumvent them by spoofing request headers. More robust solutions like Google's Play Integrity API and Apple's DeviceCheck provide attestation mechanisms that allow native applications to verify the authenticity of client devices or applications making requests [**?** **?**]. Unfortunately, these powerful mechanisms aren't inherently compatible with the PWA architecture and require native integration components, effectively leaving standalone PWAs without access to such sophisticated attestation layers.

While PWAs and TWAs offer performance and UX benefits, their openness to webview-based duplication remains a critical vulnerability.

3. Problem Definition and Attack Modeling

This section provides a formal definition of the PWA replication attack and considered attack model of the SPARE framework.

3.1. Problem Definition

Suppose a PWA $\mathbb{P}$ provides services to a set of clients $\mathcal{C}$ through a weblink $\mathbb{W}$. An adversary $\mathbb{A}$ has developed a malicious WebApp, denoted as $\mathbb{P}^A$, which replicates the functionality of $\mathbb{P}$. The malicious WebApp redirects users of $\mathbb{P}^A$ to $\mathbb{W}$, allowing the adversary to gain financial benefits or access confidential information.

A client $c \in \mathcal{C}$ is considered legitimate if they access $\mathbb{W}$ via $\mathbb{P}$, and malicious if they access $\mathbb{W}$ through $\mathbb{P}^A$. To prevent unauthorized access to $\mathbb{W}$ from applications other than $\mathbb{P}$, we propose embedding fingerprinting information in the form of query parameters in the URL of $\mathbb{W}$.

Initially, the encrypted UTC time of the client's device, $\bar{t}$, is used as the sole query parameter. A client c can access resources from a dedicated server $\mathbb{S}$ using a URL of the form $w\,?\,\bar{t}$, where $w \in \mathbb{W}$, $c \in \mathcal{C}$, and $\bar{t} = \text{enc}(\text{deviceTime}(c))$. Here:

- enc(.) encrypts data using a symmetric encryption key shared between $\mathbb{P}$ and $\mathbb{S}$.
- deviceTime(c) returns the UTC time of the client c's device.

The server $\mathbb{S}$ will deny access if it receives a request for w without any query parameters. For a URL with valid syntax, $w\,?\,\bar{t}$, $\mathbb{S}$ decrypts $\bar{t}$ to obtain the client's UTC time, $t = \text{dec}(\bar{t})$. If the difference between t and the server's current UTC time, t_s, exceeds a predefined threshold T (set to 1 minute in our experiment), the server denies the request. This mechanism reduces the attack surface since static weblinks $\mathbb{W}$ used by adversaries in $\mathbb{P}^A$ are ineffective without appropriate query parameters.

However, an adversary could access a valid URL ($w_a \in \mathbb{W}$, with query parameters $\bar{t_a}$) by interacting with $\mathbb{P}$ through their browser, as explained in Section 2 *(refer to the specific subsection)*. Despite this, such URLs would become invalid since by the time a malicious client accesses $\mathbb{W}$ through $\mathbb{P}^A$, the server's UTC time t_s would significantly differ from $t_a = \text{dec}(\bar{t_a})$ (i.e., $t_s - t_a \gg \text{T}$).

In our work, we consider a more sophisticated adversary. This adversary maintains a database $\mathbb{D}$ that is updated every Δt (where $\Delta t < \text{T}$) by retrieving valid URL from $\mathbb{P}$ through their browser. When a client opens the malicious WebApp $\mathbb{P}^A$, the WebView in $\mathbb{P}^A$ loads the most recently updated link from $\mathbb{D}$ instead of a static URL, effectively bypassing the defense. Consequently, $\mathbb{S}$ cannot differentiate between malicious traffic and legitimate traffic. Therefore, developing an effective strategy to mitigate such attacks is crucial.

3.2. Attack Model

An attack model is a structured framework that defines an adversary's assumptions, capabilities, and goals in a security scenario. It is used to analyze and simulate potential threats to a system systematically. Here, we provide the characteristics of the attack and the adversary, focusing on the following components.

3.2.1. Attack Technique

When implementing authentication for Progressive Web Applications (PWAs) via query parameters, developers face significant security challenges due to browser history accessibility. This analysis examines potential attack vectors against query parameter-based authentication systems.

The adversary creates a malicious WebApp that mimics the legitimate service provider. This malicious app redirects users to official weblinks while allowing the adversary to gain financial benefits or access sensitive data. To counter this, the legitimate provider embeds encrypted timestamps in the URLs to validate access requests. The server verifies whether the timestamp falls within an acceptable time window, blocking outdated or missing timestamps to prevent unauthorized access.

Fundamental Vulnerability

The core vulnerability exists because:

- PWAs in Trusted Web Activities (TWAs) leave traces in browser history.
- Query parameters used for authentication are visible in these URL histories.
- Webview applications can load these URLs with their authentication parameters.

A more advanced adversary maintains an updated database of the valid URL by frequently extracting fresh timestamps from the legitimate app through sniffing on the browser cache. This database is updated at intervals shorter than the allowed time window for validation, ensuring the adversary always serves recent, valid links to users. As a result, the server cannot distinguish between legitimate and malicious access, effectively bypassing the defense mechanism.

Step-by-Step Attack Process

Initial Access: An attacker first obtains the legitimate application and installs it on one or more devices. This provides access to the structure of the authentication query parameters, the pattern and frequency of parameter updates, and any observable encryption or encoding patterns.

History Extraction: The attacker develops a system to extract browser history from devices running the authentic application. This involves creating a background service that monitors Chrome/browser history, focusing specifically on URLs matching the target PWA's domain, and extracting full URLs including authentication query parameters.

URL's Cloud Infrastructure: A cloud-based collection system is established. Devices with the authentic app installed upload extracted URLs to a centralized database. The system tags each URL with metadata, including a timestamp and the source device. These URLs are then indexed and stored for distribution.

Dynamic Webview: The attacker creates a webview-based clone application that connects to the cloud repository upon launch, requests a fresh, unused authentication parameter, dynamically constructs and loads the URL with this parameter, and reports back when the parameter has been used.

Mark URL: To avoid detection through duplicate usage patterns, each authentication parameter is marked as "used" after distribution. The system prioritizes the newest parameters to maximize validity periods and implements load balancing across multiple harvesting devices.

Scaling the Attack: To overcome defensive measures like time-based thresholds, multiple harvesting devices are deployed to generate a continuous stream of valid parameters. The attack infrastructure may distribute different harvesting devices across various IP addresses and locations. Each harvesting device maintains normal usage patterns to avoid triggering anomaly detection.

Threshold Circumvention: When faced with per-device thresholds, the attacker expands their device pool, potentially using emulated devices. Parameters are harvested from multiple devices to stay under individual threshold limits. The system may also incorporate device rotation strategies to distribute harvesting activities.

3.2.2. Attack Goal

The primary goal of the attack is to bypass the server's security measures and ensure that users of the malicious WebView app can access the weblinks without interruption. Suppose the most recently updated weblink in the database is $w\,?\,\bar{t_{a'}}$, where $t_{a'} = \mathrm{dec}(\bar{t_{a'}})$. The attack will succeed only if the time difference between the server's current time t_s and $t_{a'}$ satisfies $t_s - t_{a'} < \mathrm{T}$; otherwise, the malicious clients will be denied access.

3.2.3. Attack Assumptions

The proposed framework considers a set of assumptions.

- **Assumption I:** The adversary continuously interacts with the legitimate PWA application to obtain the latest valid URL. The firewall of the legitimate server application cannot detect and block the adversary. Adversaries may employ various spoofing techniques to evade detection.
- **Assumption II:** The adversary-created WebView application can dynamically load content by fetching the URL from the adversary's database.
- **Assumption III:** The adversary is aware of the user validation process, including analyzing query parameters and the fact that the query parameter encrypts the timestamp when a client initiates a request. However, the encryption key remains unknown to the adversary.
- **Assumption IV:** The adversary's database has sufficient capacity to process and handle requests from all malicious app clients attempting to retrieve a valid weblink.

3.2.4. Adversarial Attributes

The adversarial attributes encompass knowledge, capability, and accessibility. While we have already outlined these attributes in the assumptions, we categorize them as follows.

Knowledge: The adversary knows the encryption mechanism and that the query parameter contains an encrypted timestamp but does not possess the encryption key. This characterizes the attack as a gray-box attack.

Accessibility: The adversary retrieves valid weblinks by extracting them from the adversary's copy of the legitimate application's network traffic or browser cache. The adversary can obtain the legitimate weblink containing the encrypted query parameters by sniffing on the browser cache or intercepting the application's communications.

Resources: The adversary maintains a database and possesses a copy of the malicious app. The database does not need to store all previously retrieved weblinks; instead, it only retains the most recently updated weblink at any given time. However, sufficient bandwidth is required to ensure all malicious app users can access the database content simultaneously.

4. Technical Details

In this section, we present the technical details of the SPARE framework. We begin by outlining the problem scope and proposing an initial defense strategy (see 4.1). Next, we identify attack vectors that bypass this initial defense strategy (see 4.2). We then enhance the defense strategy to develop SPARE, which is designed to counter the identified attack vectors (see 4.3). Finally, we explore an advanced attack technique aimed at bypassing SPARE and assess its effectiveness (see 4.4).

4.1. Problem Scope and Initial Defense Strategy

Figure 1 illustrates the overall customer segmentation, divided into two groups—one using the legitimate application and the other redirected to a malicious application that closely resembles the original. The malicious mobile application utilizes a WebView to load content using the same URL as the original PWA. The adversary or malicious app developer can easily obtain the original weblink used by the PWA by analyzing network packets or inspecting browser history while interacting with the original PWA as a regular user. Consequently, the malicious application acts as a man-in-the-middle between the customer and the server, enabling the adversary to integrate additional features into the application to achieve their attack objectives. To mitigate such an attack, we propose a primary defense strategy that leverages query parameters in the weblink. Instead of allowing unrestricted access to the server, we introduce an application firewall that filters incoming traffic based on the content of the query parameter. The security enhancement strategy is depicted in Figure 1, where the PWA retrieves the current UTC time from the user's device, encrypts it using a symmetric encryption technique such as AES, and embeds the resulting ciphertext containing the encrypted timestamp in the query parameter.

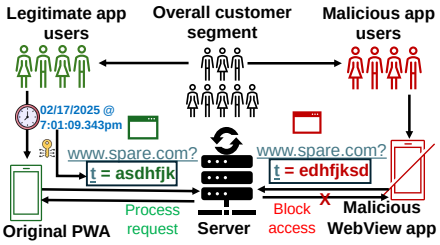


Figure 1: Overview of the problem scope and initial defense strategy.

The server validation process is further illustrated in Figure 3, where the server analyzes the query parameter before processing client requests. Since the server possesses the decryption key, it can decrypt the query parameter to extract the timestamp indicating when the request was initiated. After retrieving the timestamp, the server compares it with its UTC time. While there may be slight differences due to communication delays, even for legitimate requests, this delay remains within an acceptable threshold, assumed to be 1 hour in our problem scenario (as shown in Figure 3). If the time difference is within the allowed threshold, the server considers the request legitimate, processes it, and responds to the client. However, in Figure 1, the malicious app user is denied access because, by the time they attempt to reuse a copied URL, the time difference far exceeds 1 hour, making the request invalid.

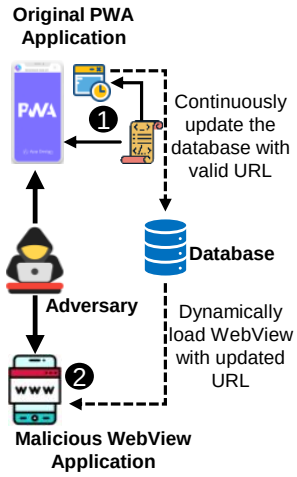


Figure 2: Demonstration of the exploitation of the initial defense strategy.

4.2. Exploiting the Initial Defense Strategy

As we explore potential defense strategies to prevent PWA replication attacks, we aim to identify attack vectors capable of penetrating the system to assess the robustness of the proposed defense. The attack model for this exploit is detailed in Section 3.2. Figure 2 illustrates an adversary's steps to evade the proposed initial defense strategy. In this scenario, we consider an adversary who develops a malicious WebApp and maintains a copy of the original PWA application. The adversary executes ❶ an automated script that continuously interacts with the original PWA, extracting the most recent valid URL from the browser history on their device. This updated URL is then stored in a database. Any client using the malicious application will have their ❷ WebView dynamically populated with the URL retrieved from the database rather than a static URL.

The validity of the URL is determined by the server running the firewall, which assesses whether the URL was generated within an acceptable timeframe. The success of this intrusion hinges on two factors: the server threshold T and the speed at which the database updates by interacting with the original PWA. If the adversary's system can retrieve and distribute updated URLs quickly enough, the attack may bypass the server's validation mechanism, thus compromising the effectiveness of the initial defense strategy.

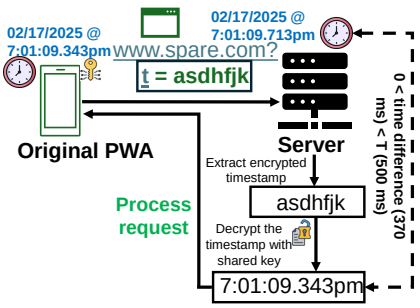


Figure 3: Overview of the server operation.

4.3. Advancing the Defense

We assume the attacker can successfully evade the initial defense strategy, as discussed in Section 4.2. To strengthen the defense, the SPARE framework introduces an enhanced mechanism that incorporates the device ID into the query parameter from the PWA application instead of solely embedding an encrypted timestamp. To further reinforce security, we introduce a new threshold, R, designed to restrict an infeasible number of requests from the same client. This threshold is determined based on the activity patterns of the most active legitimate users, ensuring that normal usage remains unaffected while preventing excessive, automated, or replicated requests from a single source. Hence, with this enhanced defense mechanism, a client c can access resources from the dedicated server $\mathbb{S}$ using a URL of the form $w?\overline{td}$, where $w \in \mathbb{W}$, $c \in \mathcal{C}$, and $\overline{td} = \text{enc}(\text{deviceTime(c)} \mid\mid \text{deviceID(c)})$. Here, deviceTime(c) and deviceID(c) represent the timestamp and device ID of the client's device, respectively. We use the symbol ($||$) to denote concatenation.

The attack strategy that successfully bypasses the initial defense will fail against this advanced defense mechanism since all malicious app users rely on URLs generated from a single compromised device. As a result, the server will flag and block repeated requests originating from the same device ID once they exceed the defined threshold R. The attack strategy that successfully bypasses the initial defense will fail to evade this advanced defense mechanism since all malicious app users rely on URLs generated from a single compromised device. As a result, the server will flag and block repeated requests originating from the same device ID once they exceed the defined threshold R in a single day. This enhanced security measure significantly limits the effectiveness of the defense, as the adversary can no longer distribute valid URLs among multiple users without being detected.

To further strengthen the defense, we introduce an additional firewall rule that detects simultaneous requests from the same device. Such behavior indicates potential replication, a key characteristic of the attack. The system will flag it as a suspicious activity if multiple requests are detected from the same device ID within a short window. We propose blocking access for device IDs that exhibit repeated, simultaneous request patterns, effectively neutralizing the attack by preventing the mass distribution of valid URLs.

Our defense methodology implements a dynamic approach to secure application access through query parameters. Upon initial URL launch, a unique query parameter is generated and transmitted to the frontend. The frontend verifies this parameter and forwards it to the backend for authentication purposes. A limitation exists where this query parameter is only generated upon the first launch. When the application is not running in the background, the system prompts users to close and reopen the application, creating a suboptimal user experience.

To address this limitation, we store the query parameter in local storage when the user launches the application. The request payload includes a "resubmit" parameter, which is set to true when a user is already in the application and initiating a server request. This eliminates the need for users to repeatedly close and reopen the application. Server-side handling of the resubmit value requires careful implementation, as the query parameter remains unchanged, potentially accessing existing data. Database updates include refreshing the user's last request timestamp with each interaction. The system maintains a rolling database architecture that compares current request times against previous request timestamps. Each request increments an access counter associated with the device ID. When a user reaches the defined threshold, they

receive a notification to pause requests for a predetermined period, after which their access count resets, allowing them to regain access privileges.

4.4. Advanced Attack Technique

To thoroughly evaluate the robustness of our enhanced defense mechanism, we now consider an advanced attack strategy aimed at circumventing the newly introduced security measures. Figure 4 illustrates this proposed exploit. In the previous attack scenario, the adversary could not spoof (i.e., modify) the device ID while storing updated URLs from the original PWA, as the URL generation process involved encryption. Consequently, distributing a single valid URL across multiple malicious app users is no longer viable, as exceeding the threshold R requests from the same device ID would be flagged as an anomaly.

To overcome this restriction, the attacker adapts their strategy as follows:

1. The adversary interacts with ❸ multiple copies of the original PWA from different physical devices. Each device independently retrieves valid URLs in real time.
2. These URLs are then continuously updated in the ❹ centralized database accessible by the malicious applications.
3. Instead of distributing URLs randomly among malicious app users, the attacker employs a ❺ round-robin distribution strategy. This approach systematically cycles through available URLs, ensuring that no single device ID exceeds the daily request threshold R.

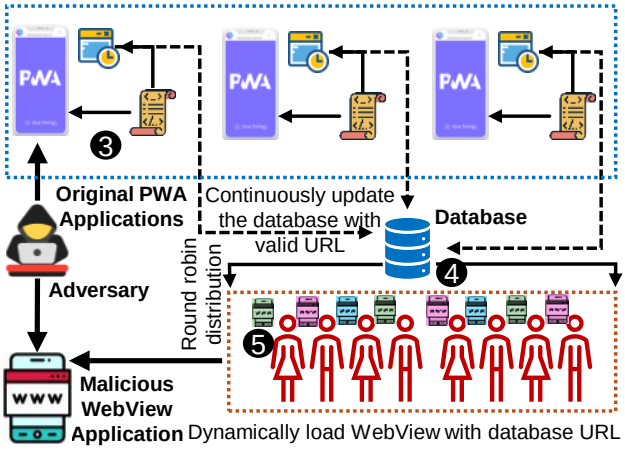


Figure 4: Demonstration of the advanced exploit.

By leveraging multiple devices and distributing URLs efficiently, the adversary reduces the likelihood of detection and denial from the server due to excessive request activity from a single device. This method represents a more sophisticated and scalable approach to bypassing the SPARE defense mechanism.

5. Case Studies

In this section, we present empirical case studies to demonstrate how the SPARE framework operates in analyzing the security of a PWA application. We begin with a benign scenario in Case Study 1, followed by subsequent case studies that showcase how the proposed defense strategy performs against attackers with varying levels of knowledge and skill. To enhance clarity and explainability, we use simplified numerical values in the case studies. The original data is used in the following section (Section 6) for the formal evaluation of the SPARE framework.

5.1. Case Study 1: Benign Scenario

Here, we present a numerical study of a benign, attack-free scenario. We assume that 20 users actively use the PWA, each generating between 25 and 45 daily requests. To manage bandwidth usage and mitigate the risk of denial-of-service attacks, we introduce a threshold to limit excessive request rates. Table 1 presents the percentage of successful and failed requests at various per-day device access thresholds for authenticated users. The table shows that as the daily threshold increases, the number of successful requests (requests processed by the server) also rises. When the threshold is set to 45, the success rate reaches 100%, as no user exceeds 45 requests per day in this scenario.

Table 1: Case Study of a Benign Scenario Demonstrating Percentage of Successful/Failed Requests with Varying Per Day Device Access Threshold for an Authenticated User

Number of PWA Users	Minimum Number of Requests/User-Day	Maximum Number of Requests/User-Day	Total Requests/Day	Average Number of Requests/User-Day	Threshold	Number of Successful Requests (Percentage of Successful Requests)	Number of Failed Requests (Percentage of Failed Requests)
20	25	45	666	33.3	30	600 (90%)	66 (10%)
					35	664 (99.7%)	2 (0.3%)
					40	666(100%)	0 (0.0%)
					45	666 (100%)	0 (0%)

5.2. Case Study 2: Amateur Attacker

An amateur attacker is unaware of the complete encryption process. The attacker attempts to create a WebView application by simply copying the URL used in the original application, i.e., `http://spare.com`, and distributing the malicious app. However, users of this malicious application will be denied access, as the requests sent to the server will lack the required valid query parameters.

5.3. Case Study 3: Naive Attacker

A naive attacker lacks understanding of the internal mechanisms of the system. Specifically, they are unaware of the encryption-based validation used in the URL query parameters. However, unlike the amateur attacker, the naive attacker recognizes the importance of query parameters for accessing resources from the server. Suppose the attacker creates a malicious WebApp that visually mimics the original PWA and distributes it to a group of users. The attacker hardcodes a URL such as `http://spare.com?id=hrdqiuayf378ary389qhqueaid78` into the malicious application. The attacker may initially access the URL from their own browser using the original application. However, once the URL has been used, any subsequent attempts to access it—such as from the distributed malicious apps—will fail. This is because the server tracks previously used URLs for resource requests. Each query parameter includes an embedded device ID and timestamp indicating the request's initiation. If the same link is reused, the server detects that a request from the same device ID and timestamp has already occurred, signaling a replication attempt. As a result, the WebView in the malicious application fails to load the intended content, rendering the attack ineffective.

We now consider a scenario where the attacker does not connect the original application to the internet, preventing any requests from reaching the server. However, since the query parameter generation occurs on the front-end of the original PWA, a valid URL can still be generated locally and copied directly from the browser, even without an internet connection. As a result, the attacker can obtain a valid URL for distribution. This URL remains valid for a threshold duration, denoted as T, which we assume to be 1 hour in this case. Suppose the attacker distributes the malicious WebApp, including the generated URL, to a user within this 1-hour window. In that case, the user can access the application for that day, provided they do not exceed the per-day device access threshold (e.g., 35 requests). However, the server will detect the replication attempt if multiple users attempt to access the application using the same URL. All requests from the second and subsequent users will be denied in such cases. Therefore, through this method, the naive attacker can at most enable a single user to access the malicious WebApp for one day. The session will expire on subsequent days, and any attempt to reuse the same URL will result in denied requests.

5.4. Case Study 4: Moderate Attacker

Moderate attackers possess a deeper understanding of the system than naive or amateur attackers. They can identify the limitations of previous attack strategies and adapt accordingly. Specifically, a moderate attacker knows that a single device ID cannot exceed the per-day request threshold R without being flagged by the server. To circumvent this restriction, the attacker employs multiple physical devices—five in our case—to interact with the original PWA and retrieve valid, encrypted URLs in real time. These URLs are stored in a centralized database accessible to all instances of the distributed malicious application. The attacker also knows the user request distribution and, based on this information, continuously generates valid URLs using the five attacker devices.

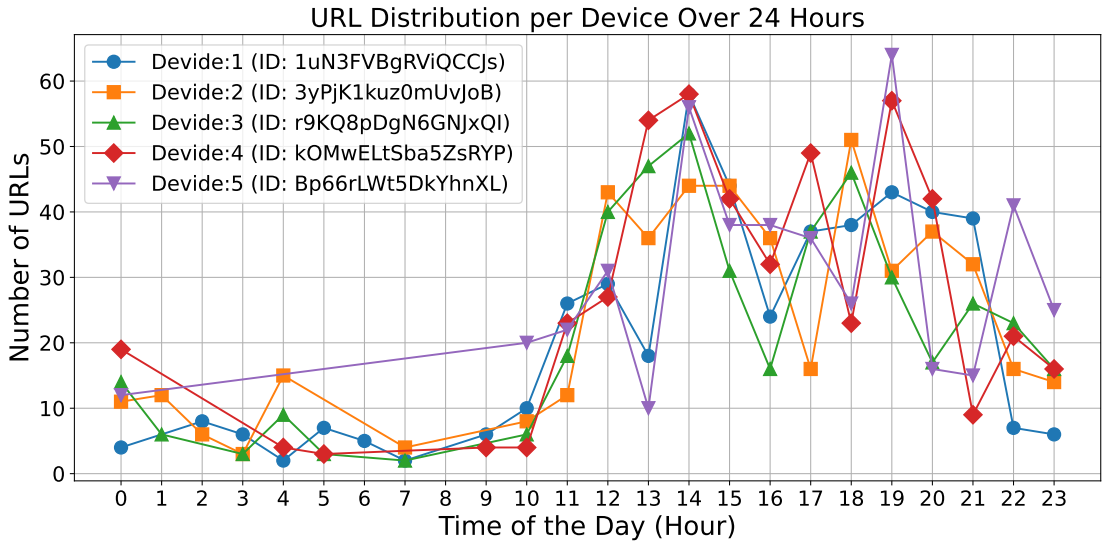


Figure 5: Number of URLs generated at a certain time by an attacker device.

Figure 5 illustrates the number of URLs generated by the attacker's devices at different hours of the day. These URLs are then distributed to 20 malicious app users. However, the attacker applies a random distribution strategy, which results in the same URL (with identical timestamp and device ID) being assigned to multiple users. Consequently, the server detects these replication attempts and denies the requests. In our case study, 707 requests were made, averaging 35.35 requests per user. These were handled by the five attacker devices, which received an average of 134.80 requests each. However, as the per-device threshold was set to 30 requests, most of the requests exceeded the limit. As a result, only 150 requests (21.2%) were successful, while the server denied the remaining 557 requests (78.8%).

5.5. Case Study 5: Sophisticated Attacker

We now consider a more advanced attacker who employs a round-robin distribution strategy to rotate the use of valid URLs among malicious app users. This approach avoids triggering request anomalies by ensuring that no single URL or device ID exceeds the per-device request threshold. In this case, the attacker immediately deletes a URL from the centralized database once it has been distributed to a user, preventing reuse and reducing the likelihood of detection. As a result, the success rate of the attack improves compared to the previous scenario. However, given that the per-device threshold remains set at 30 requests, many still exceed the limit. Out of a total of 716 requests, 300 (41.9%) were successfully served, while the remaining 416 requests (58.1%) were denied by the server.

6. Evaluation

This section presents the findings from evaluating the proposed PWA defense against an advanced attack strategy. We present the results of SPARE's evaluation by addressing the following research questions.

RQ1 What is the impact of the device access threshold for the benign scenario? (Section 6.1)

RQ2 What are the framework findings in assessing the proposed attack impact with variable system parameters? (Section 6.2)

RQ3 What are the framework findings in assessing the proposed attack impact with variable attacker's capability? Section 6.3)

Table 2: Benign User Request Statistics

No of User	Threshold	Total Requests	Successful Requests	Failed Requests	Failed Percentage
100	30	3558	2942	616	17.31
100	35	3558	3263	295	8.29
100	40	3558	3477	81	2.28
200	30	6937	5832	1105	15.93
200	35	6937	6441	496	7.15
200	40	6937	6796	141	2.03
300	30	10347	8803	1544	14.92
300	35	10347	9577	770	7.44
300	40	10347	10013	334	3.23

6.1. System Evaluation in Benign Scenario

6.1.1. Benign User Performance Analysis

We evaluated system performance under various benign user population sizes and request thresholds. The dataset includes configurations for 100, 200, and 300 users, each using a unique device, with thresholds set at 30, 35, and 40.

Figure 6 illustrates the relationship between the failure percentage and threshold across different user counts. As expected, increasing the threshold significantly reduces the failure rate. For instance, with 100 users, the failure percentage drops from 17.31% at a threshold of 30 to just 2.28% at a threshold of 40. A similar pattern is observed for 200 and 300 users, although the absolute failure rates are slightly higher due to increased system load.

This trend highlights the importance of appropriate threshold tuning to maintain system reliability. Lower thresholds are more restrictive and can lead to higher rejection rates, while higher thresholds may allow more benign traffic through but may reduce sensitivity to anomalous behavior. Therefore, selecting a suitable threshold must balance accuracy and user experience.

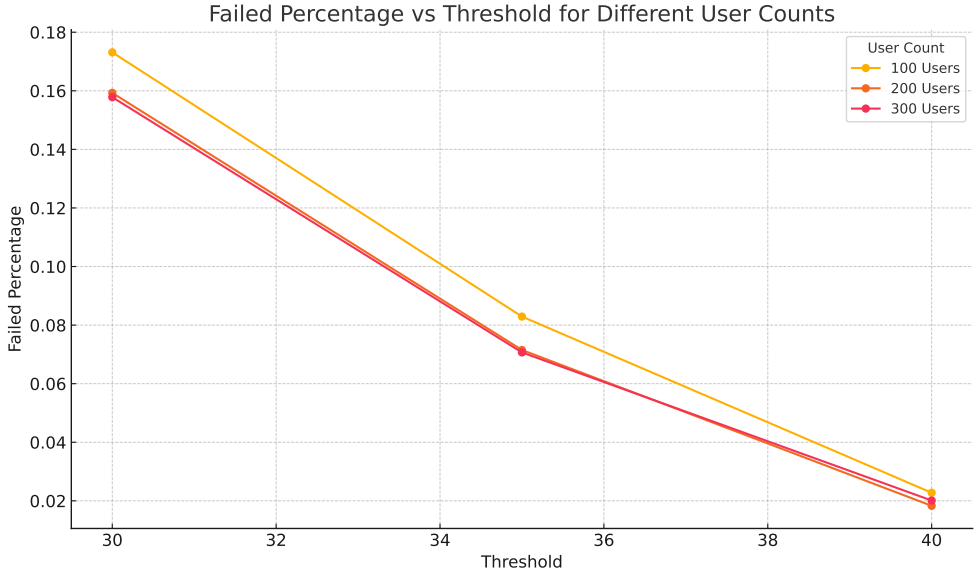


Figure 6: Failed Percentage vs Threshold for Different User Counts

Table 3: Malicious User Request Statistics (Failure Rates in Percentage)

No of User	Attack Devices	Threshold	Total Requests	Successful Requests	Failed Requests	Failed Percentage
100	10	30	3524	300	3224	91.49%
100	20	30	3580	600	2980	83.24%
100	30	30	3466	870	2596	74.90%
100	10	35	3524	350	3174	90.07%
100	20	35	3580	700	2880	80.45%
100	30	35	3466	1013	2453	70.77%
100	10	40	3524	400	3124	88.65%
100	20	40	3580	798	2782	77.71%
100	30	40	3466	1152	2314	66.76%
200	10	30	6922	300	6622	95.67%
200	20	30	6859	600	6259	91.25%
200	30	30	7040	900	6140	87.22%
200	10	35	6922	350	6572	94.94%
200	20	35	6859	700	6159	89.79%
200	30	35	7040	1050	5990	85.09%
200	10	40	6922	400	6522	94.22%
200	20	40	6859	800	6059	88.34%
200	30	40	7040	1197	5843	83.00%
300	10	30	10413	300	10113	97.12%
300	20	30	10388	600	9788	94.22%
300	30	30	10331	900	9431	91.29%
300	10	35	10413	350	10063	96.64%
300	20	35	10388	700	9688	93.26%
300	30	35	10331	1050	9281	89.84%
300	10	40	10413	400	10013	96.16%
300	20	40	10388	800	9588	92.30%
300	30	40	10331	1200	9131	88.38%

6.2. Attack Impact Evaluation by Varying System Parameters

6.2.1. Malicious User Scenario Analysis

Table **??** presents a detailed overview of system performance under malicious conditions across 27 unique configurations. The dataset systematically varies three parameters: the number of users (100, 200, 300), the number of attack devices (10, 20, 30), and the rejection threshold (30, 35, 40). Each row represents a unique test case simulating coordinated malicious behavior with different levels of device distribution and system sensitivity.

Across all configurations, the failure percentage—representing the proportion of rejected requests—remains significantly high, validating the system's capacity to identify and block suspicious traffic. Several key patterns emerge:

- **Impact of Threshold:** Increasing the threshold from 30 to 40 consistently reduces the failure percentage. For example, with 300 users and 10 devices, the failure rate decreases from 97.12% at threshold 30 to 96.16% at threshold 40. This suggests that higher thresholds make the system slightly more tolerant, allowing more requests to pass.
- **Impact of Attack Device Count:** For a fixed number of users and a given threshold, increasing the number of attack devices results in a lower failure percentage. For instance, under threshold 35 with 100 users, failure drops from 90.07% (10 devices) to 70.77% (30 devices). This indicates that a more distributed attack—using more devices—is harder to detect, as device reuse becomes less obvious.
- **Impact of User Volume:** As the number of users increases, total request volume scales proportionally. However, the system maintains robust detection performance. At thresholds 30 and 10 attack devices, failure rates are 91.49%, 95.67%, and 97.12% for 100, 200, and 300 users, respectively—demonstrating strong scalability in anomaly rejection.

This comprehensive simulation highlights the importance of balancing detection aggressiveness (via thresholds) against the nature of attack strategies (e.g., spoofing fewer vs. more devices). It also validates that even under increased attack volume, the system preserves a high rate of malicious traffic rejection.

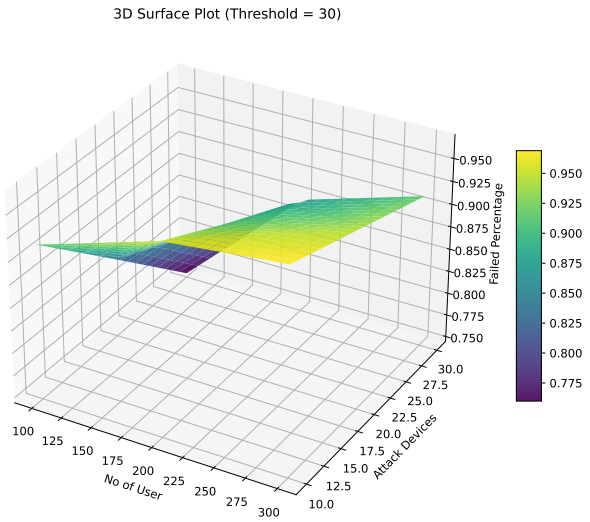


Figure 7: 3D surface plot showing failed percentage as a function of number of users and attack devices at a fixed threshold (Threshold = 30).

6.2.2. 3D Surface Analysis of Failure Behavior

Figure 7 presents a 3D surface visualization of the failed percentage as a function of the number of users and attack devices, keeping the threshold fixed at 30. This plot provides a comprehensive view of how the system behaves under varying load and attack distribution.

The surface reveals two key trends:

- **Increase in users leads to higher failure rates:** As the number of users increases, the surface rises along the failure percentage axis. This is intuitive—more users generate more requests, thereby increasing the system's workload and the likelihood of rejection when device reuse is high.
- **Increase in attack devices reduces failure percentage:** Conversely, increasing the number of attack devices leads to a reduction in failure percentage, flattening the surface. This reflects the system's challenge in detecting distributed attacks, where malicious activity is spread across many devices, lowering the signal per device.

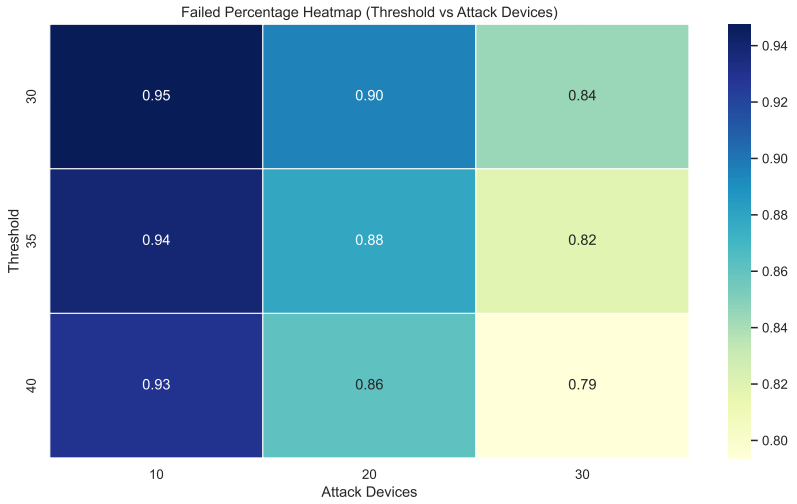
The curvature of the surface plot effectively illustrates the nonlinear interaction between user volume and device diversity. When few devices are used with many users, the failure percentage peaks sharply. However, as device count increases, even high user loads lead to lower rejection rates.

This 3D perspective complements the 2D heatmaps and reinforces earlier observations: centralized attacks are easier to detect, while distributed strategies demand more sophisticated countermeasures.

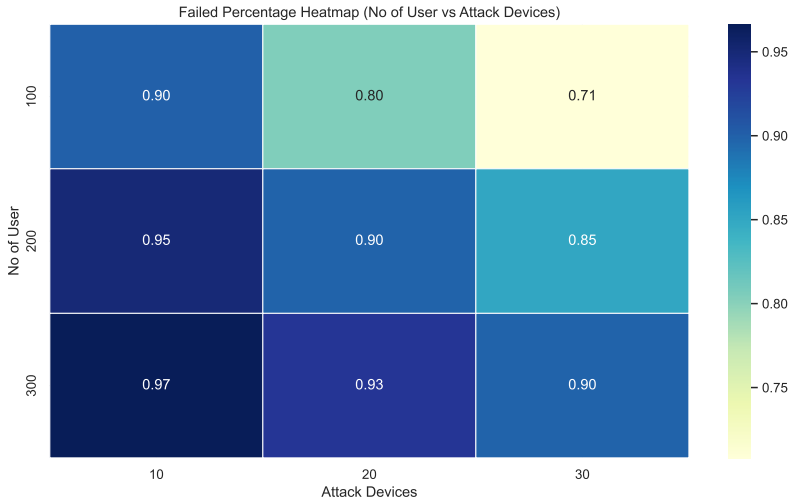
6.2.3. Heatmap-Based Analysis of Failure Patterns

Figure 8 illustrates the variation in failed request percentage across three different parameter dimensions using heatmaps. Each subfigure presents how failure behavior changes based on the interaction of two parameters while holding the third constant.

(a) Threshold vs Attack Devices



(b) No of Users vs Attack Devices



(c) No of Users vs Threshold

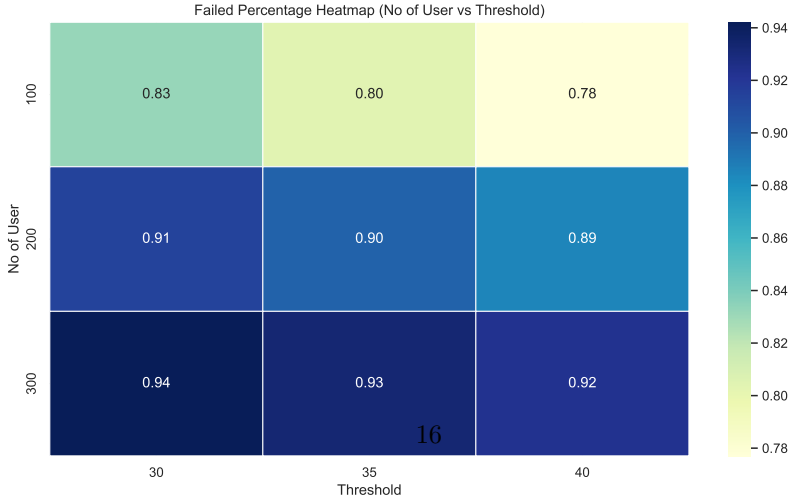


Figure 8: Heatmaps showing failed percentage across various parameter configurations: (a) threshold vs attack devices, (b) number of users vs attack devices, and (c) number of users vs threshold.

(a) Threshold vs Attack Devices.. In subfigure (a), we observe the relationship between the number of attack devices and the system's threshold setting. The failure percentage decreases as the number of attack devices increases, which is expected since spreading malicious activity across more devices reduces the likelihood of detection due to decreased device reuse. Conversely, raising the threshold makes the system more permissive, also reducing failure rates. Notably, the combination of low threshold and few attack devices results in the highest failure rates—demonstrating the system's effectiveness under tight security conditions and low device diversity.

(b) Number of Users vs Attack Devices.. Subfigure (b) explores the impact of user count and attack device count on failure percentage. As the number of users increases, the system handles more requests, but still maintains high rejection rates when the number of attack devices is small. For example, 300 users with only 10 attack devices show a very high failure percentage due to dense device reuse. As attack devices increase, failure rates decrease across all user sizes, highlighting the challenge of detecting distributed attacks where each device mimics fewer users.

(c) Number of Users vs Threshold.. In subfigure (c), we analyze the system's sensitivity to different threshold values across varying user counts. For all user sizes, increasing the threshold results in a lower failure percentage—again indicating the trade-off between strictness and permissiveness. However, even at higher thresholds, the system maintains reasonably high detection when the number of users is large, showing its scalability and robustness.

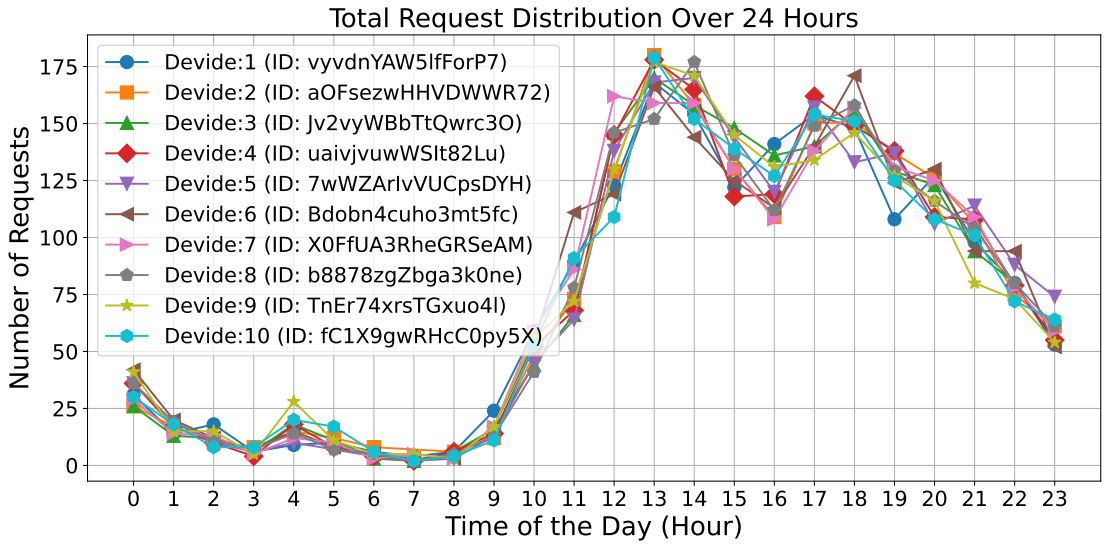


Figure 9: Number of URLs generated at a certain time by the attacker devices.

6.3. Attack Impact Evaluation by Varying Attacker's Capability

6.4. Comparison Between Malicious and Benign Users with the same number of requests

6.4.1. Impact of Defense Technique on First Error Detection

Figure 10 illustrates a comparative analysis of benign and malicious users based on their total request count and the position of their first rejected request. This analysis is conducted under a system threshold of 30 requests per user.

Figure 11 presents a 3D scatter plot that visualizes the separation between malicious and normal (benign) request scenarios across three key parameters: number of users, number of attack devices, and system threshold.

The distribution of points in the plot reveals distinct patterns:

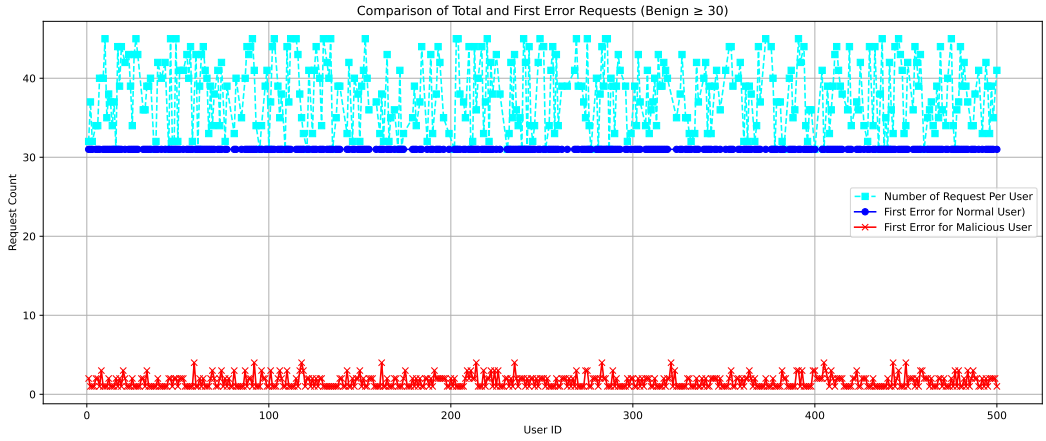


Figure 10: Comparison of total requests and first error occurrences for benign and malicious users. (Threshold = 30)

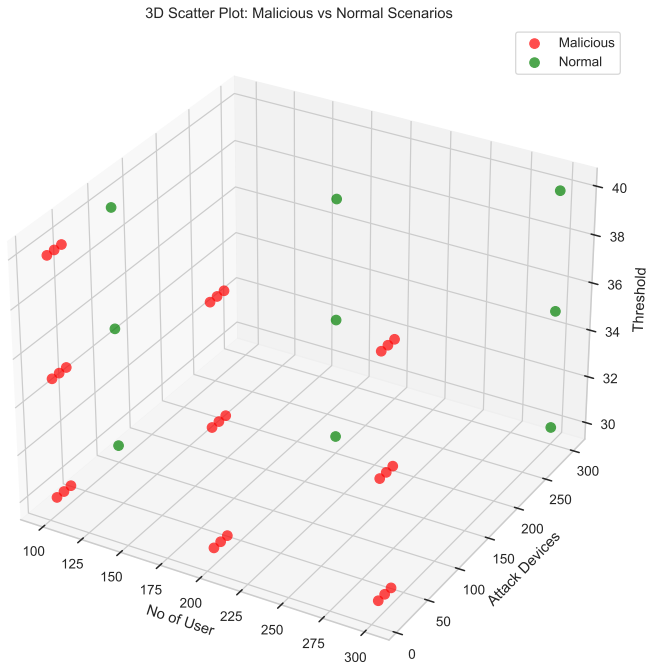


Figure 11: 3D scatter plot comparing failure patterns in malicious and normal scenarios based on number of users, attack devices, and threshold settings.

- **Normal users** are generally clustered at higher thresholds and exhibit lower variability across the number of attack devices—consistent with expected, rule-abiding usage patterns.
- **Malicious users**, on the other hand, appear across a wider range of device counts and are often associated with lower thresholds. Their spatial dispersion in the plot reflects the unpredictability and aggressiveness of adversarial behavior.
- The **clear spatial separation** between malicious and normal points in this 3D space demonstrates the potential for using multivariate analysis or machine learning classification techniques to distinguish between the two groups.

This visualization supports the earlier analytical results, showing that malicious activity not only deviates in terms of failure rates and first errors but also forms recognizable patterns across system-wide parameters. It provides a strong foundation for future development of intelligent anomaly detection systems using multi-dimensional data.

A key feature of our Progressive Web App (PWA) is the integration of a custom defense technique, designed to proactively detect and mitigate malicious behaviors before they reach the predefined threshold. The observed data supports the effectiveness of this approach:

- **Benign Users:** Benign users, operating within normal behavioral bounds, generally experience their first error exactly at the threshold limit. Their request patterns are consistent, and the system permits all legitimate interactions up to the point where the static threshold policy applies.
- **Malicious Users:** Malicious users, however, face rejections significantly earlier—often within the first few requests—well before reaching the threshold of 30. This behavior is a direct outcome of our defense strategy.
- **Effectiveness of the Defense:** The early rejection of malicious requests demonstrates the proactive nature of our defense mechanism. Rather than relying solely on volume-based limits, the PWA incorporates behavioral and contextual indicators to trigger defense actions. This allows the system to prevent abuse at an early stage, minimizing potential impact while preserving usability for legitimate users.

These results confirm that our defense-aware PWA can effectively distinguish between benign and adversarial request patterns, leading to timely detection and mitigation of malicious activities.

7. Discussion and Future Work

The performance of the proposed Progressive Web App (PWA) defense mechanism is evaluated by comparing the behavior of benign and malicious users across two key metrics: *failed percentage* and *position of first error*.

Failed Percentage Analysis.. The failed percentage metric, derived from both the Normal and Malicious scenario datasets, quantifies the system's ability to reject undesirable requests. In benign scenarios, failure percentages remain low and are tightly bound to the predefined system threshold. As the threshold increases, benign request rejections decrease accordingly, confirming that legitimate users are rarely flagged incorrectly.

In contrast, malicious users exhibit significantly higher failure percentages across all configurations. Even at relatively high thresholds and increased attack device distributions, malicious users consistently face failure rates ranging from 67% to over 95%. This demonstrates the robustness of the defense system against adversarial behavior, even under high-load and distributed attack conditions.

First Error Behavior.. Figure 10 highlights another layer of differentiation by visualizing when the first rejection occurs per user. For benign users, first errors typically occur at or just beyond the defined threshold (e.g., 30 requests), indicating consistent and acceptable user behavior. Malicious users, however, encounter their first rejection within the early phase of request sequences—often within the first few interactions.

This early detection of malicious activity is a direct result of the implemented defense technique within the PWA. The system does not rely solely on count-based thresholds but utilizes behavioral patterns, device ID reuse, and timing anomalies to proactively identify suspicious users. As such, many malicious users are rejected long before they reach the nominal threshold, while benign users are allowed to operate with minimal friction.

In the current system, the request threshold is set manually (e.g., 30 requests), which may not be optimal across different applications or usage patterns. In future work, we plan to explore machine learning techniques to dynamically determine appropriate thresholds based on user behavior and historical request data. This would allow the system to adaptively respond to varying usage contexts.

Additionally, we aim to investigate alternative secure communication techniques for PWAs, beyond the current encrypted request model. Exploring more robust mechanisms such as token-based authentication, device attestation, or end-to-end encryption could further strengthen the system's defense against spoofing and unauthorized access.

8. Dataset Details

To simulate realistic app usage patterns, we developed a synthetic dataset generator that models user-device interactions over a 24-hour period. The system simulates encrypted user requests, timestamps, and device identifiers, ensuring behavioral diversity and cryptographic consistency across users and time.

8.1. User and Device Modeling

The dataset consists of N users, each generating a random number of requests drawn from a uniform distribution between 25 and 45 requests. Each user is assigned a set of devices from a pool of D unique device identifiers. Device assignment follows a round-robin mechanism to simulate multiple users sharing overlapping device sets, mimicking real-world shared environments or spoofed device identities.

8.2. Temporal Distribution of Requests

To capture realistic app usage behavior, the hourly request distribution follows a normalized, empirically-inspired pattern.

As illustrated in Figure 12, activity is minimal during late-night hours (12AM–5AM), increases significantly in the morning, peaks between 9AM–11AM, and gradually declines in the evening. Request timestamps are sampled by weighting hourly selection with this usage distribution and then randomly choosing a second within the selected hour.

8.3. Encrypted Parameter Generation

Each request contains a timestamp and a device identifier, concatenated and formatted as `timestamp@PWAdeviceID`. These raw data strings are encrypted using AES in CBC mode with a predefined 16-byte secret key and initialization vector (IV). This ensures confidentiality and simulates real-world obfuscation of query parameters. Encryption and decryption validity is asserted for every entry to maintain consistency.

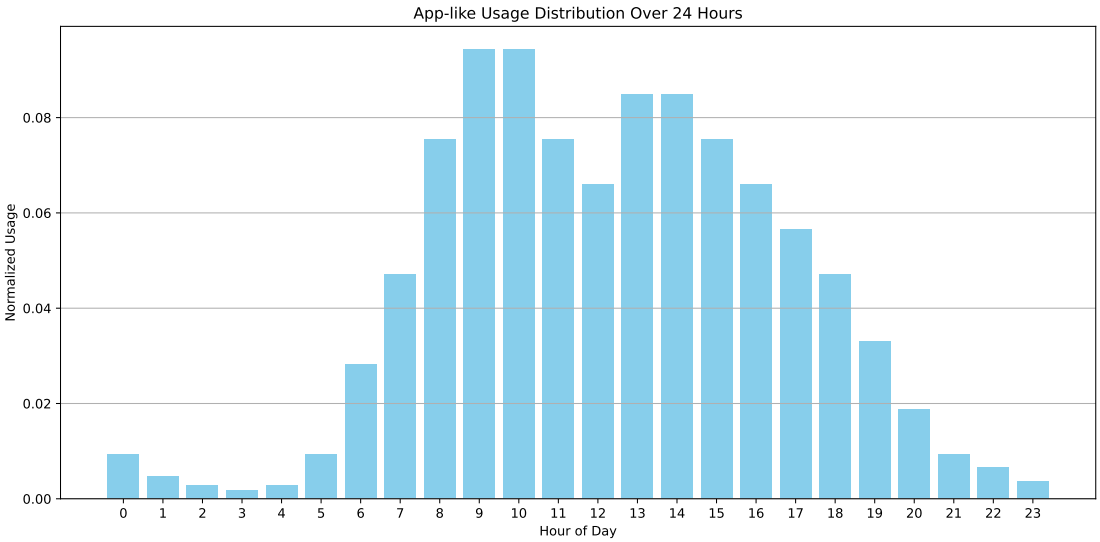


Figure 12: User Request Distribution.

8.4. Benign and Malicious User Dataset Configuration

We generate two distinct types of datasets: one for benign users and another for malicious users, each designed to reflect different behavioral patterns in request generation and device usage.

Benign Users: For the benign user scenario, we synthesize datasets for three population sizes: 100, 200, and 300 users. Each benign user is assigned a unique device, ensuring a one-to-one mapping between user and device identifiers. Request generation for each user follows the app-like hourly usage distribution discussed previously (see Figure 12), representing natural, human-driven app interaction patterns. This setting serves as the baseline for evaluating normal system behavior.

Malicious Users: In contrast, the malicious user datasets are designed to simulate adversarial behavior with potential device spoofing or identity duplication. For each user population size (100, 200, 300 users), we simulate attacks using 10, 20, and 30 unique device identifiers respectively. These device identifiers are cyclically reused across users, simulating scenarios where attackers attempt to impersonate multiple users using limited physical devices. As with benign users, the request timestamps for malicious users are also sampled using the same temporal distribution model to ensure comparability.

Comparative Objective: This two-tiered generation approach enables a robust evaluation of system performance under normal (benign) and adversarial (malicious) conditions. By varying the number of users and attack devices, we can analyze the impact of user density and device reuse on system accuracy, failure rates, and detection capabilities.

9. Conclusion

The combined analysis of failed percentage and first error positions confirms the effectiveness of the defense technique integrated into our PWA. While benign users experience smooth interaction and only face errors at policy-defined limits, malicious users are accurately and swiftly identified based on early behavior deviations.

These results validate two critical outcomes:

1. The system maintains high usability for legitimate users with minimal false positives.

2. The defense technique effectively detects and disrupts malicious activity early, improving system resilience against attacks.

Such a dual capability—balancing user experience with proactive security—demonstrates that intelligent request monitoring, beyond static thresholds, is vital for defending web applications against modern attack strategies.

1. Pride and Prejudice in Progressive Web Apps: Abusing Native App-like Features in Web Applications [2].

References

[1] Progressive web application market size, `https://www.emergenresearch.com/industry-report/progressive-web-application-market`, accessed: 2025-01-20 (2021).

[2] N. Seifo, M. Robertson, J. MacLean, K. Blain, S. Grosse, R. Milne, C. Seeballuck, N. Innes, The use of silver diamine fluoride (sdf) in dental practice, British Dental Journal 228 (2) (2020) 75–81.